Codex 自动剪辑：从安装到第一条成片

视频配套实操资料 · 2026-10-12

这份资料讲的是我们这套科普视频工作流：由 Codex 写脚本、调用工具、组织画面和时间线，你负责确认内容、试听声音、看样片和提修改。它不是 Codex 内置了一个传统剪辑软件，也不是装一个 Skill 就能保证一键出精品。

建议先做一条 30-60 秒的短科普，跑通后再做长视频。下面以 Apple Silicon Mac 为主线；Windows 用户请先看第 2 节，不要照搬 MLX 安装命令。

1. 先看懂：到底要装什么

工具	在这套流程里负责什么	怎么获取
Codex	理解要求、写代码、调用其他工具、修改作品	安装官方客户端或使用官方 CLI
HyperFrames	把 HTML 画面组织成时间线，预览并渲染视频	插件或 Skills，加上 Node.js CLI
Fish Audio	把脚本生成口播；需要时克隆本人或授权声线	注册在线服务，不用下载语音大模型
Qwen3-ASR-1.7B	将最终口播转成文字，供核对字幕	本地模型 + 推理程序
Qwen3-ForcedAligner-0.6B	把核对好的文字对到音频时间	本地模型 + 推理程序
HTML / CSS	排版、字号、颜色、图片位置	浏览器原生技术，不是要下载的 App
SVG	图形、箭头、连线、示意图	浏览器原生技术，由 Codex 写
GSAP	图形和文字的动画、镜头、转场	JavaScript 代码库
Rough.js	手绘线条、涂鸦图解	可选 JavaScript 代码库
FFmpeg	转音频格式、调音、裁切、合并、检查成片	本地命令行工具

人负责选择和验收，Codex 负责执行。 软件名不用全部背下来，知道它们在什么时候接棒就行。

流程：需求 → 脚本 → 口播 → 转写与对齐 → 画面与时间线 → 配乐与检查 → 视频文件。

2. 电脑和费用：先选对路线

Apple Silicon Mac

我们实际采用的是 MLX 路线：在苹果芯片上运行 Qwen 的 8bit 字幕模型。MLX-Audio 的要求和安装方式见其项目文档。

- 16GB 内存可作为首次尝试的参考配置，不代表所有项目的最低要求。

- 首次需要联网下载软件和模型，下载完成后字幕推理可本地进行；Codex 服务和 Fish Audio 仍需要相应网络连接。

- 留出空间存模型、素材、渲染缓存和多版视频，不要只按最终 MP4 的大小估算。

Windows / Intel Mac

画面制作和字幕推理是两件事。不要在 Windows 上安装本文的 MLX 版 Qwen。可以让 Codex 按 Qwen 官方项目核对 PyTorch 路线和你的硬件，再决定本地部署或经过你同意的其他方案。本文没有验证 Windows 的完整链路，也不提供未经测试的万能安装命令。

哪些可能花钱

Codex 的账号与用量按你使用的服务计算；Fish Audio API 口播按账号和服务规则计算；素材、音乐及可选生图也可能有费用。HyperFrames 的本地开源渲染与这些服务的计费不是一回事。不要把某个模型名称里的 free 理解为整套流程永久免费。HyperFrames 快速开始说明了本地渲染与可选服务的区别。

3. 下载、安装和第一次检查

3.1 Codex：先打开一个专用文件夹

从官方桌面入口进入下载和使用说明。官方页面当前已跳转到 ChatGPT 桌面文档，入口名称可能与视频里不同，以官网为准；安装后选择 Codex 的工作入口。终端用户也可看官方 CLI 说明。

新建一个专门的制作文件夹，例如 codex-video-demo，在客户端里打开它并开始任务。素材和输出都放在这个文件夹中，不要让新手教程从整个桌面或文档目录随意抓文件。

先粘贴这段：

> 帮我检查这个视频工作区的环境。检查操作系统和芯片、Node.js、npm、Python、FFmpeg、HyperFrames，以及是否有对应 Skills。只检查，不批量安装，不生成配音。给出缺失项、官方来源和安装方案，已安装的不要重复装；不要修改其他项目。

3.2 Node.js、Python、FFmpeg：让工具能运行

官方下载入口：

- Node.js：优先选择官网标注的 LTS；安装器包含 npm，npx 用于运行工具。

- Python：字幕脚本的运行环境。首次部署可先使用 Python 3.12，让 Codex核对依赖兼容性。

- FFmpeg：官网列出不同系统的获取方式。已有 Homebrew 的 Mac 可以用 brew install ffmpeg，不需要为本教程重复安装一套 Homebrew。

让 Codex 先报版本：

```
node --version
npm --version
python3 --version
ffmpeg -version
```

能显示版本只是安装检查；下一步还要试出声音、字幕和一个小视频，才能算流程能用。

3.3 HyperFrames：Skill 与 CLI 要分清

Skill 是告诉 Codex 怎么制作的说明书；CLI 是实际执行预览、检查和渲染的程序。只下载网页上的 SKILL.md，不等于已经拥有能渲染视频的环境。安装路线依据HyperFrames 官方快速开始。

如果客户端已有 HyperFrames 插件，先使用插件自带的 Skills 和启动方式，不要叠加安装多套。如果没有，告诉 Codex：

> 在这个工作区安装 HyperFrames 的制作能力。先检查是否已有插件或 Skills；有就复用，没有则按官方方式运行 npx hyperframes skills update。安装后列出可用 Skills 的名称，并用 CLI 帮助和环境检查确认可运行。不要自动转写，不要下载 Whisper。

终端方式的核心命令：

```
npx hyperframes skills update
npx hyperframes doctor
```

按提示选择 Codex 对应的位置；安装完成后必要时重新打开任务，让新 Skills 被识别。Codex 的本地 Skills 位置和加载规则以官方 Skills 文档为准，不要凭别的 Agent 的目录猜。

官方 HyperFrames Skills 中，入口 hyperframes 负责路由制作任务；core 管结构与时间线，animation 管动效，audio 管混音，cli 管预览和渲染，media-use 管素材与声音。实际名称和安装集合以安装结果为准，日常只需要明确要求“使用 HyperFrames”，不用每句话都点名所有 Skills。本资料不依赖我们本机的私人声线预设或额外私有 Skill。

创建一个横屏小项目时，可在专用工作区内运行：

```
npx hyperframes init my-video --resolution landscape
```

本教程不在 init 时传入音频或视频，以免走到框架自带的转写流程。先把空项目建好，后面再由 Codex 导入我们准备的口播和字幕。命令及参数依据官方 CLI 文档；遇到不同版本时先看 --help，不要照抄不支持的参数。

3.4 GSAP、Rough.js、SVG：不是再开三个软件

它们直接用在 HyperFrames 的画面代码里。让 Codex 检查项目依赖，已有就复用，没有才补充；不要强行把三套动画引擎混在一起。

如果项目采用 npm 管理，可以按需安装：

```
npm install gsap
npm install roughjs
```

GSAP 官方安装文档提供 npm 和浏览器脚本等方式；Rough.js 官网有手绘图形示例。HTML / CSS / SVG 则不用额外下载。

4. Fish Audio：让文字变成你的声音

从 Fish Audio 官网注册。API 的创建和调用见官方快速开始，声音创建见官方克隆说明。

这里有两种用法，先选一种：

1. **网页先做声音**：在网页创建或选择声线，试听，再导出口播音频，交给 Codex。适合第一次跑通。

2. **API 接到流程里**：创建 API key，让本地脚本从安全配置读取，Codex 按确认过的文字调用。适合后续重复制作。

克隆参考用你本人或获得授权的原始录音。选一段人声清楚、语气自然、没有背景音乐的内容，同时准备与录音一致的文字。服务对格式、长度的要求以当前官方页面为准，不要拿带音乐的成片或上次 AI 合成的声音再做克隆参考。

先做 10-15 秒试听，重点听：是否像本人、重音是否自然、英文和缩写有没有读错。通过后再生成全稿。字幕可以保留标准英文拼写，TTS 输入可单独写更容易读对的发音，例如字幕写 Qwen3，口播写“千问三”。

API 接入时给 Codex 这段：

> 帮我编写 Fish Audio 口播调用脚本，API key 从 FISH_API_KEY 或明确约定的本地私密配置读取，不能写进源码、文档、日志或聊天。先说明这个读取方式在桌面客户端的子进程里是否生效，再生成短试听；不要批量生成。我提供的原始参考录音和核对文字用于本人克隆，或者使用我已经创建的 voice ID。

环境变量不是在任意终端 export 一下就会传到已经打开的桌面客户端。如果采用 CLI，可从同一个终端启动 Codex；桌面模式则让脚本明确接入专用安全存储或本地私密配置，私密文件不要纳入 Git 或资料包。

保存原始生成音频，再生成最终口播。例如需要稍快一些，可用 FFmpeg 的 atempo 调整，不要给已经加速过的文件再加速：

```
ffmpeg -i narration-raw.mp3 -af atempo=1.12 -ac 1 -ar 44100 narration-final.wav
```

1.12 是我们当前科普声线的制作偏好，不是人人都要照用。最终字幕和画面必须以 narration-final.wav 的实际声音为基准，而不是原始音频的时长。

5. Qwen 字幕：先转写，再校对，再对齐

这两个模型不能混成一个功能：ASR 回答“说了什么”，ForcedAligner 回答“这些字什么时候说”。我们用的是 MLX-Audio 的 Qwen 实现。

模型入口：

- Qwen3-ASR-1.7B-8bit

- Qwen3-ForcedAligner-0.6B-8bit

Hugging Face 页面里的模型权重不是双击就能开的 App，需要通过推理程序加载。已有缓存就复用；没有缓存时，首次加载才需要下载。不要手动下载很多零散文件后猜文件夹结构。

在 Apple Silicon Mac 的工作区创建独立 Python 环境，避免污染其他项目：

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install mlx-audio
```

让 Codex 使用这个环境，并先检查它是否支持下面两个精确模型 ID。如果缺少支持，让它核对项目文档和包版本，不要自动换成 Whisper。

给 Codex 的字幕提示词：

> 使用最终口播 narration-final.wav，先用 mlx-community/Qwen3-ASR-1.7B-8bit 转写，保存 transcript-draft.txt。列出与批准脚本不同的地方供核对，专名保留标准写法；确认后保存 transcript-checked.txt。再用 mlx-community/Qwen3-ForcedAligner-0.6B-8bit 将核对文字与同一条音频对齐，输出 words.json 和 captions.srt。复用缓存；没有模型时先说明下载需求。不擅自换识别模型。最后试听核对几个关键词和首尾字幕，不能只报告文件已生成。

要理解底层调用，可把下面的逻辑交给 Codex 写成带输入输出参数的脚本，不需要自己整套重写：

```
from mlx_audio.stt import load

asr = load("mlx-community/Qwen3-ASR-1.7B-8bit")
draft = asr.generate("narration-final.wav", language="Chinese")
print(draft.text)

# 人工核对后，将文字保存为 transcript-checked.txt
aligner = load("mlx-community/Qwen3-ForcedAligner-0.6B-8bit")
checked_text = open("transcript-checked.txt", encoding="utf-8").read()
aligned = aligner.generate(
  "narration-final.wav", text=checked_text, language="Chinese"
)
```

这是调用结构示例，不是完整字幕导出程序。长音频通常需要分段，让 Codex按当前实现的限制切段、对齐，并把每段时间加回整条音频的起点；不要直接拼一堆从 0 秒开始的时间戳。

字幕时间对齐后，再合并为容易读的短句，而不是把每个字都作为一条字幕。最终检查字幕有没有遮住图形、输入框和重要操作。若之后修改口播速度、删句或加停顿，重新做相关对齐。

6. 文件怎么放，才不会越做越乱

建议让 Codex 建立下面的目录。名称可以改，但输入、源代码与输出要分开。

```
my-video/
  brief.md            需求
  script.md           批准的口播稿
  storyboard.md       每句对应的画面
  assets/             真实截图、视频、图片
  audio/              最终口播、BGM、音效
  captions/           校对文字、时间戳、SRT
  compositions/       各场景画面代码
  index.html          主时间线
  renders/            样片、预览、最终视频
  README.md           启动方法与版本说明
```

API key、私人原始参考录音、登录信息都不放进公开资料包。需要分享项目时另外做一份去敏副本。不要拿当前工作目录直接一键打包上传。

每次明显修改另存版本，例如 sample-v1、full-v1、full-v2，不要覆盖已经认可的成片。配音参考、口播文件和音乐文件也要分开，防止音色参考混入 BGM。

7. 第一条视频：照这个顺序跑

第一步：只让它定脚本

先选一个简单问题，例如“API key 和 URL 有什么区别”。这只是练习主题，也可以换成你熟悉的内容。

> 做一条 16:9 横屏、约 45 秒的中文科普视频，解释【主题】，面向【观众】。用大白话，开头直接提出具体问题。现在只给口播稿，不制作整片、不生成配音。要用哪些真实截图、哪里适合图解，在口播稿后另列。技术事实需要核对来源，未核实的内容不要编。

拿到脚本后删废话，确认专名和事实。此时改文字最便宜。

第二步：做短口播，再做全稿

> 这版脚本确认。先按授权声线生成开头 10-15 秒试听，语气自然、有重点，不用播音腔。听感确认后再分段生成全稿，保留原始文件和最终口播文件。记录实际时长，后续以声音安排画面。

第三步：字幕处理

使用第 5 节的提示词。检查英文名、数字、断句，核对后再对齐。不要让 TTS 的情绪控制标签出现在字幕里。

第四步：让每句话都有对应画面

> 按最终口播和对齐时间戳写分镜。每句列出起止时间、讲解重点、视觉主体、动效和素材来源。真实截图用于说明在哪里操作，SVG 用于解释关系。不要全片固定左文右图，也不要把口播全文贴在屏幕上。每段至少有明确的视觉变化，但不要为了动而乱晃。

例子：讲 URL 时，先放大真实地址栏，再切图解展示请求去哪里；讲字幕对齐时，显示同一段波形和逐词点亮；讲 GSAP 时，让刚才静止的箭头真的延伸。说到某个工具，画面就展示它的作用，不只摆一个名字。

第五步：先做 15-20 秒样片

> 使用 HyperFrames 制作开头 15-20 秒样片。画面风格【填写】，大标题手机上可读，字幕有独立安全区。HTML/CSS 排版，SVG 画图，GSAP 动画；只有手绘场景才加 Rough.js。按词时间戳触发重点，不按估算的脚本时长硬排。加入有节奏的纯音乐和轻音效，口播优先清晰。给出本地预览和项目位置。

先看这四项：内容看懂了没有、声音像不像自己、画面是否有变化、音乐是否舒服。样片没过，不急着把同样的问题铺满两分钟。

第六步：批准后扩展为全片

> 按已经认可的样片标准做完整视频，不换风格。每个章节保持相同的字号、字幕安全区和动效密度。保留素材来源；没有真实操作视频的地方明确做功能示意，不冒充录屏。先给完整预览，我确认后再导出。

8. 音乐、修改和导出怎么配合

音乐先单独选，不满意就不要硬混

准备你有权使用的纯音乐，把授权条件或来源一起保存。有口播的知识视频，优先试没有人声、鼓点不过于抢戏、能循环或覆盖全长的音乐。视频里某首歌好听，不代表分离出来就取得了使用授权。

> 先给我几首有节奏的纯音乐试听，适合知识科普，不要人声、不要过软。说明来源和使用条件。选定后再混入视频：开头无口播时音乐可突出，口播出现时适当降低，结尾自然收束，不要中间突然断开。

常见做法是把整体混音的约 -14 到 -16 LUFS 作为试听起点，再根据人声与音乐的实际动态调整；不是抖音的保证通过标准。既检查响度，也听清人声和音效，不能只把总音量推高。

用“对象 + 修改 + 不动什么”来提反馈

> 只改 12-18 秒这一段：标题大约放大 30%，截图只留对应输入框，字幕移到底部安全区。口播不变，后面时间线不变。另存新版本，给我该段预览。

> 口播不变，把 BGM 稍微提高；遇到强调词和关键动画时加一个轻音效，不要每个字都打一下。检查有没有突然变响或断音乐。

比“高级一点”“有感觉一点”更容易改对的，是描述主体大小、对齐位置、出现时间、希望保留的内容。不要同时换脚本、声线、音乐和风格，再猜是哪项变好了。

预览和导出命令

在包含 index.html 的项目目录运行，具体参数以当前 CLI 帮助为准：

```
npx hyperframes check
npx hyperframes preview
npx hyperframes render --quality delivery --output renders/final-hd.mp4
```

这些命令的用途见HyperFrames CLI。分辨率由项目画布和渲染配置共同决定，文件名写 HD 不会自动变高清。首次先导 1080p / 30fps，不需要一开始就追 4K。

导出提示词：

> 当前预览确认。导出 1920×1080、16:9、30fps MP4，优先兼容播放器和社交平台。导出后检查尺寸、时长、帧率、视频和音频轨；抽查每个场景与转场，完整解码检查错误，确认没有字幕遮挡、无关工具角标或声音中断。报告实际检查结果和最终文件路径，旧文件保留。

至少亲自完整看一遍最终 MP4；浏览器预览能播放，不代表导出文件的声音和画面就一定正确。

9. 常见卡住的地方

现象	先检查什么
只给建议，没生成文件	是否在能操作项目的工作任务里；是否明确要求创建项目和交付路径
装了 Skill 还不能剪	Skill 是否被加载；Node、CLI、浏览器和 FFmpeg 是否可用
Fish 提示 401 / 402	key 是否能被脚本读取、是否有效；账号余额或权限，别重建画面
模型反复下载	缓存位置、模型 ID、离线配置；不要先删缓存重来
Qwen 安装失败	芯片、Python 环境、包版本；Windows 不用 MLX 路线
口播对、字幕错	ASR 的专名错误；先校对文字，再用同一音频重新对齐
后半段音画逐渐错位	分段时间是否累加；是否对齐后又加速、删音频或加停顿
浏览器有图，导出没图	本地路径、资源加载、动画是否支持跳转到任意时间检查
截图字看不清	只展示相关区域，放大；不要把整个桌面缩成一个小窗
画面像 PPT	是否只换静态页；让图形在讲解时完成比较、连接、展开等动作
音乐盖住口播	分别检查轨道和混音；降低或换 BGM，不只调整总音量

遇到错误就把报错和当前项目路径交给 Codex，让它先诊断、少量修改、再验证，不要反复整套重装。

10. 一段完整开工提示词

> 我要用 Codex 跑通自动剪辑，先做一条【时长】的【横屏/竖屏】科普，主题【主题】，风格【风格参考】。工作只在当前专用文件夹里进行。
>
> 先检查 Node.js、Python、FFmpeg、HyperFrames 与对应 Skills，已有就复用，缺失时说明官方安装方法，不批量重装。本人克隆口播使用 Fish Audio，参考只用授权的原始录音；API key 通过明确的安全配置读取，不写源码、不输出到聊天。
>
> 字幕采用 Qwen3-ASR-1.7B 转写、校对后用 Qwen3-ForcedAligner 对齐；Apple Silicon Mac 使用对应的 mlx-community 8bit 模型，复用缓存。若当前系统不支持这条路线，先说明并给方案，不擅自换模型。
>
> 制作顺序：需求与脚本 → 我确认 → 短口播试听 → 全稿口播 → 校对字幕与时间戳 → 分镜 → 15-20 秒样片 → 我确认 → 完整预览 → 我确认 → 高清导出。每一阶段交付真实文件和路径。
>
> 画面用 HyperFrames、HTML/CSS、SVG 和 GSAP，手绘场景按需用 Rough.js。真实素材与功能示意区分；我提供【素材】。每个重要讲解都有对应视觉动作，标题和字幕手机可读。加入有节奏的授权纯音乐与轻音效，音画同步。修改另存版本，不能覆盖认可成片。

11. 分享前再做一个小检查

资料包和项目源码是两回事。发给别人前确认没有 API key、Cookie、私人语音参考、账号截图、私人聊天或无权分发的音乐。可以分享方法与提示词，不默认分享你所有制作素材。

首次给朋友试用，目标是“他自己的电脑能做出一条短样片”，不是承诺几分钟做完任意风格的长视频。我们已经有本地制作案例，但这份新版资料没有在全新 Mac 或 Windows 上逐条安装测试；遇到版本差异，按官方文档和环境检查修正。

官方入口索引

- Codex / 桌面官方入口 · Codex Skills
- HyperFrames 仓库 · 快速开始 · CLI 文档
- Fish Audio · API 快速开始 · 克隆声音
- Qwen 官方项目 · MLX-Audio · Qwen MLX 用法
- Qwen ASR 8bit · Qwen ForcedAligner 8bit
- Node.js · Python · FFmpeg
- GSAP 安装 · Rough.js

入口核对日期：2026-10-12。工具更新较快，版本、页面名称和账户权限以实际官方信息为准。